PROMOIQ: narrative-aware promo discovery for microdrama screenplays

Kuku AI Generalist assignment · Problem 1: Find the Promos · show: *I Married a Broken Soldier. He was a Duke* (50 episodes)

Why this problem and how I framed it

Problem 1 has a real input pack and an output anyone can check: episode, start and end. It also removes a bottleneck that grows with every weekly release, because today an editor watches every episode. I treated a promo as a **sequence to construct**, not a scene to score. The brief's three tests become constraints and an objective over legal cut points: a hook in the first 3 s, an unresolved ending, and sense without context. Text alone cannot prove a promo works, so the tool gives the editor a short, ranked, evidence-backed shortlist and keeps the final call human.

Assumptions (where the brief was silent)

1) Screenplay timestamps are the cut's timeline (MM:SS.ss, decimals are seconds); the editor checks each cut on video. 2) A legal cut is a camera or scene boundary that does not slice a line of dialogue or a sound cue. 3) Footage duration is the sum of segment lengths, with no transitions. 4) Episode number is story order, although several episodes retell earlier ones (E1/E4, E10/E13, E17/E18). 5) Joined segments move forward in story time. 6) The title's premise ("He was a Duke") is not a spoiler. 7) It must run free, so an offline provider is the default and LLMs are optional plug-ins.

How it works

Parse. A deterministic, case-tolerant parser reads **3,727** events (camera 1754, dialogue 1487, action 372, sound 114) from 50 episodes (102.7 min) with 0 parse errors. These layers overlap in time, so they are merged into **1,633 beats**, and a cut is only allowed at a beat boundary. Every cut is therefore a real source timestamp and never splits a line. **Score.** Each beat gets eight 0-5 features (emotion, conflict, stakes, reveal, curiosity...) from delivery cues ("Outraged, Accusing"), wording, camera and sound cues and the writers' loglines. These give a tension curve T = .30E + .30C + .25S + .15U, a hook strength at every start point and a cliffhanger strength at every end point. **Story.** A story bible covers characters, relationships with evidence and conflicts. A pair only counts as a conflict if their own lines are more hostile than warm. Conflict stages and a NetworkX graph (231 nodes, 579 typed edges) provide targeted context for each promo. **Search.** Every legal 20-60 s continuous window is enumerated exactly. Multi-segment promos come from a **constrained beam search** over 450 strong moments: joins must move forward, share a rival, ally or conflict, fit in 60 s and not retell the same moment. Partial sequences are ranked with an optimistic cliffhanger bound. This gave **3,367** candidates (2,967 continuous, 400 multi). **Objective.** Q = .20 Hook + .20 Cliff + .20 Clarity + .15 Emotion + .15 InfoGap + .10 Coherence - .15 Redundancy - .20 Spoiler (each 0-10; weights are hypotheses, set in config.py). **Check.** A Python hard validator, a cold-viewer judge that sees only the promo, an integrity judge with story context and a spoiler taxonomy, then gates. A hard failure can never be offset by a high score. **Rank.** Duplicates are found by time overlap, shared beats, same scene and calibrated MiniLM similarity. An MMR pass picks a diverse shortlist, quality orders it, boundary counterfactuals are added, and strong alternatives held back for diversity are shown to the editor.

What AI does vs. what code does

Code (deterministic)	AI (LLM when configured; offline rules otherwise)	Human
parsing, all timestamp maths, legal cut points, duration and overlap checks, search, ranking, duplicates, evidence lookup	beat scoring and labels, alias and relationship resolution, cold-viewer and integrity judging, pairwise boundary choice. Every quote the model gives is checked against the source; unverifiable claims are flagged	final pick from the shortlist and alternatives, cut check on video, weight changes

Results on the full pack (offline provider, 0 LLM calls, $0)

Run time **22 s** on a laptop. **1,243** candidates passed every hard check and gate. **2,124** were rejected: weak cliffhanger 1,765, weak hook 669, spoiler risk 92, not standalone 14, incoherent join 5; one candidate can fail several gates. The shortlist covers 6 angles (confrontation, action, romance, betrayal, family, mystery). Post first:

#	Segments (episode start-end)	Dur.	H	K	C	Why it works (generated)
1	E27 01:32.80-02:04.60	32s	7.7	9.4	8.0	Confrontation angle with Blond Nobleman and Juliana: an unresolved ending and rising emotion; conflict CATHERINE vs JULIANA continues after the cut.
2	E28 00:37.70-01:15.30	38s	9.1	7.2	9.0	Confrontation angle with Dowager Duchess and Blond Nobleman: a hard-hitting first 3 seconds and a conflict a stranger can follow; conflict CATHERINE vs JULIANA continues after the cut.
3	E24 00:32.80-01:10.50	38s	9.3	9.4	8.4	Action angle with Noblewoman and Juliana: an unresolved ending and a hard-hitting first 3 seconds.
4	E32 01:11.20-02:02.80	52s	8.4	7.7	9.0	Confrontation angle with Juliana and Catherine: a conflict a stranger can follow and a hard-hitting first 3 seconds; conflict CATHERINE vs JULIANA continues after the cut.
5	E31 01:23.20-02:02.50	39s	9.0	8.4	8.0	Romance angle with Catherine and Juliana: a hard-hitting first 3 seconds and an unresolved ending; conflict CATHERINE vs JULIANA continues after the cut.
6	E1 00:54.60-01:14.60	20s	9.1	7.3	8.0	Betrayal angle with Juliana and Lady In Gold Gown: a hard-hitting first 3 seconds and a conflict a stranger can follow; conflict GERALD vs JULIANA continues after the cut.

The full list of 10, plus multi-segment PROMO-09 (E42 → E44) and the held-back alternatives, is in outputs/final/ranked_promos.csv / .json and editorial_report.md. #1 opens on an accusation and cuts as the villain turns the scene against the heroine.

How I know it is good: baselines and blind review

Top-10 metric	PROMOIQ	A0 keyword window (no AI)	B event ranking (same features)
duration 20-60 s	100%	100%	100%
cuts on real timestamps	100%	0%	100%
no line cut mid-way	100%	10%	100%
hook pass*	100%	20%	100%
cliffhanger pass*	100%	20%	40%
standalone pass*	100%	100%	100%
spoiler flagged	0%	20%	30%
near-duplicates	0%	20%	10%
blind review overall (1-5)†	3.95	2.88	3.76
hook / cliffhanger (1-5)†	3.4 / 4.0	2.0 / 2.4	4.2 / 2.6

*Measured with PROMOIQ's own scorers, which PROMOIQ optimises, so these rows favour it partly by construction. The duration, timestamp, mid-line and duplicate rows are objective. †Blind review round 1 (on the shortlist before fix 2 below): 6 pairs and 18 shuffled clips, rated by the AI coding assistant that built the system. It is **not a human or independent rating**, and blinding was imperfect (A0 clips are exactly 30 s and visibly cut mid-line). A human rating template is included. Pairwise, PROMOIQ won 5/6. The reviewer and the model's scores agreed on 5/6.

Live LLM run: Gemini free tier, 80 calls over two days, $0

Annotation used gemini-3-flash-preview, 5 episodes per call. The judges used gemini-3.1-flash-lite, a different model. The free tier allows 20 requests per day per model, so over two days (cached answers reused) 19 candidates were LLM-judged and Baseline A covered 28 episodes. **Baseline A (LLM-only)** did better than I expected: 100% of its cuts are real timestamps and 100% don't split a line, but only 50% of its top 10 are within 20-60 s (5 run 61-86 s). PROMOIQ's value is guaranteeing 100% on all three, plus search, joins, gates and explanations. **Quotes:** 95.3% of 1,503 were verbatim and 70 were not, all flagged; spot checks found most of those stitched together from the beat's own words. **Scores:** inflated (mean stakes 4.2/5 against 0.4 offline) and only weakly aligned with the offline scorer (rank correlation 0.09 to 0.46). The code check still rejected 152 LLM-rated hooks with nothing happening in the first 3 s. **Judges:** the LLM cold viewer called 15/19 candidates not standalone. That confirms the offline proxy is too lenient, but its reasons were mostly backstory a promo should withhold. **Result:** 10 promos from 10 episodes, with no near-duplicates; one is the offline #1 (E27). Day 1's 9 judged candidates had given 5 promos from one arc, so the narrowness came from the small pool. A different judge, not a better-calibrated one; only human review can settle which is better.

Where it picks badly (real failures from this build)

1. Allies labelled as rivals. The first conflict detector made the romantic leads the show's biggest "conflict" (32 events) and printed it in promo explanations. Fixed by comparing hostility with warmth in each pair's own lines. Residual: co-conspirators who mock the heroine together are still paired. **2. Weak hooks rewarded.** The blind review rated PROMOIQ's hooks below the plain baseline (3.4 vs 4.2/5). Bare reactions ("What?") were scored as hooks; a rule was added but not re-tested blind. **3. Unrelated storylines joined.** The protagonist is in almost every scene, so early multi-segment promos joined unrelated storylines (coherence 3.7/10). Fixed by excluding the protagonist from the connection test and adding a coherence gate. **4. Diversity versus preference.** The reviewer's preferred promo (E3 rescue, 7.83) was held back as similar (0.69) to another pick. I kept the rule rather than overfit to my own review, and now surface such alternatives. **5. Open issues.** The offline clarity proxy passes every system at 100%, so it does not discriminate. It also missed an off-screen "she" that the whole promo hinges on; the LLM judge is stricter but muddles legibility with backstory. See evaluation/failure_analysis.md (19 cases, including 8 from the live Gemini run).

Scale and cost

Offline: ~20 s per 50-episode show on one laptop CPU, with no API cost. Shows are independent, so weekly releases parallelise trivially. Continuous search is linear in cut points (~33 per episode); beam search is bounded by beam width x library size. With an LLM: 1 annotation call per episode (or 5 episodes per call in batched mode: 10 calls), 1 story-bible call over loglines only (~15 KB, not 530 KB of scripts) and 2 judge calls per judged candidate. Measured free-tier run: 80 calls, 35 min over two days (mostly pacing and 503 back-off), $0; a paid tier would judge 30+ candidates per show in one pass. All calls are cached, logged and token-counted, and a stronger judge model can be swapped in with one environment variable. Once live, I would know it works by A/B testing shortlisted promos against editor-picked ones on 3-second hold rate, completion and click-through to episode 1, and by tracking how many shortlist picks editors keep.

AI tools used, and one place they got it wrong

Built with **Claude Code (Claude Opus 5.5)** for code, debugging, the blind review pass and drafting this document; the decisions and assumptions are mine to defend. The pipeline uses **sentence-transformers all-MiniLM-L6-v2** for similarity and, live, **Gemini** (3-flash-preview, 3.1-flash-lite, 3.6-flash) on the free tier. **Mistake:** the AI assistant's first conflict detector treated any two characters in a tense scene as enemies. The output looked plausible until I read the explanations against the scripts: "conflict Alexander vs Juliana continues after the cut" was attached to scenes where they are husband and wife. That is how I caught it. The same assistant also first set up the local app listening on the whole network, which would have exposed the confidential scripts; I caught it from the server log and bound it to localhost.

Limitations

Screenplay text only: no video frames, performance, music, edit or audience data. Cut points must be checked on the final cut. Scores are heuristic proxies, not audience predictions, and the weights are untuned hypotheses. The LLM paths are built and live-tested only within free-tier limits (19 LLM-judged candidates, Baseline A on 28 of 50 episodes). The offline judges are not independent of the generator. Descriptive speaker labels ("LADY IN GOLD GOWN") are not resolved to names offline. Episode order is assumed to be story order. The input format is assumed to be Kuku's screenplay layout. Prototype: GitHub repo with run steps; the app is local only because the pack is confidential.